

Project Executable Files

Date	01 October 2026
Team ID	6ab7f752d083b0e2ee72b0bf
Project Name	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Maximum Marks	3 Marks

Project Executable Files

This section lists the executable and deployable files for PocketSmart AI so the evaluator can run the solution with the provided instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes (GitHub repository and Pocket.zip)
2	README / Setup Guide (instructions to run the project)	Yes (README.md)
3	requirements.txt / package.json / dependency file	Yes (requirements.txt)
4	Database schema / seed files (if applicable)	NA (tables are created by db.create_all() from app/models.py; no seed data)
5	Environment configuration file (.env.example or similar)	Yes (.env.example)
6	Deployed application URL (if hosted)	No (runs locally)
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA (web application)
8	Dockerfile / Containerization config (if applicable)	No
9	Test files and test results	No test files; performance test results are in the Performance Testing document
10	Demo video or walkthrough (if required)	No (shown as a live demonstration)

Step 2: File / Folder Structure

```
PocketSmart-AI/
|-- run.py                # entry point: create_app() and app.run()
|-- config.py             # Config class: SECRET_KEY, DATABASE_URL, Gemini key/model, upload rules
|-- requirements.txt      # Flask, Flask-SQLAlchemy, Flask-Login, google-genai, python-dotenv, Pillow, Werkzeug
|-- .env.example          # template for GEMINI_API_KEY, GEMINI_MODEL, SECRET_KEY, DATABASE_URL, FLASK_DEBUG
|-- .gitignore            # .env, .venv, __pycache__, instance, uploads, pocketSMART.db
|-- README.md             # setup, testing, troubleshooting, production notes
|-- uploads/              # temporary outfit images (deleted after each request)
`-- app/
    |-- __init__.py       # create_app(): db, login manager, blueprints, db.create_all()
    |-- auth.py           # /register, /login, /logout
    |-- main.py           # /, /dashboard, /planner/<type>, /history/<id>, /api/health
    |-- ai.py             # Gemini client, prompt, JSON parsing
    |-- models.py         # User and Recommendation tables
    |-- templates/        # base, index, login, register, dashboard, home, party, jewelry, _result, history_detail
    `-- static/           # css/style.css, js/app.js
```

Step 3: Deployment / Access Details

Hosted / Deployed URL	Local only: http://127.0.0.1:5000
Login Credentials (Demo)	No fixed account. Open /register, create an account (name, email, password of at least 8 characters) and log in with it.
Platform / Hosting Provider	Local machine (Flask development server); source hosted on GitHub; Gunicorn (gunicorn "run:app") recommended for deployment
Repository Link	https://github.com/santhoshcreator/pocket-smart-AI.git
Demo Video Link	Not recorded; the project is demonstrated live

Step 4: Run Instructions

Step-by-step instructions to run the project locally:

1. Install Python 3.11 or newer and Git, then run `git clone https://github.com/santhoshcreator/pocket-smart-AI.git` and open a terminal in the project folder (the one with `run.py`).
2. Create and activate a virtual environment: `python -m venv .venv`, then `.venv\Scripts\activate` (Windows) or `source .venv/bin/activate` (macOS / Linux).
3. Install dependencies: `pip install -r requirements.txt`
4. Create a Gemini API key in Google AI Studio, copy `.env.example` to `.env` and set `GEMINI_API_KEY` (also `GEMINI_MODEL` and `SECRET_KEY`).
5. Start the server: `python run.py`
6. Open <http://127.0.0.1:5000> in a browser, register a user, log in and try the Home, Party and Jewelry planners.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Recommendations link to platform search pages; prices are AI estimates, not live prices.	Verify price and availability on the platform; official APIs are planned.
2	Response time and quota depend on the Gemini API and a valid key is required.	Add the key to <code>.env</code> before the first run; change <code>GEMINI_MODEL</code> if the model is unavailable.

S.No	Known Issue / Limitation	Workaround / Status
3	No CSRF protection, rate limiting or HTTPS in the development setup.	Planned hardening before any public deployment (see Scalability & Future Plan).